LevelUp Cloud Trainings
Sateesh Pabbathi | Microsoft Certified Trainer (MCT)

PySpark RDD

Module 02 — Key-Value Operations

Expert Class Notes

reduceByKey | groupByKey | aggregateByKey | filter | distinct | map | sortBy | Broadcast | 3 Use Cases | Assignment

reduceByKey	groupByKey	aggregateByKey	Use Cases
Aggregate by key	Group & iterate	Two-phase agg	3 Real Scenarios

www.levelupedu.net

ABOUT THE AUTHOR

Microsoft Certified Trainer | Data Engineering Educator | 22+ Years Industry Experience

Sateesh Pabbathi
MCT
www.levelupedu.net

Sateesh Pabbathi

Microsoft Certified Trainer (MCT)

With 22+ years in data and cloud technologies, Sateesh is the founder of LevelUp Cloud Trainings — India's premier data engineering platform. He has trained 5,000+ students in PySpark, Databricks, Microsoft Fabric, Azure, GCP and AWS.

These RDD expert notes follow his class session verbatim. Every concept is explained the way he teaches — with analogies, visuals, step-by-step internals, and real interview questions.

Expertise

- Microsoft Fabric (DP-600) | Apache Spark & PySpark
- Databricks | Azure Data Factory | GCP & AWS
- SQL Server | Python | Big Data & Hadoop Ecosystem

"I don't just teach technology — I build careers."

5,000+	22+	10+
Students Trained	Years Experience	Technologies Taught

Module 1: RDD Key-Value Operations — The Big Picture

In this session we cover the HEART of RDD programming: key-value pair operations. These are the building blocks of every distributed data processing algorithm. Master these five operations and you can solve 90% of real-world Big Data problems.

> **Sateesh Says 💡**
>
> *Think of key-value pair RDDs like a school register. The key is the student's roll number, the value is their score.*
>
> *reduceByKey says: add up all scores for each student. groupByKey says: collect all scores for each student into a list.*
>
> *aggregateByKey says: compute something MORE complex — like average — in two phases to avoid moving all data at once.*
>
> *Every data engineering job at Flipkart, HDFC, TCS uses these exact patterns every single day!*

Topics Covered in This Module (from your class notebook)

1. reduceByKey — Aggregate values per key using a binary function
2. groupByKey — Group all values for each key into an iterable
3. aggregateByKey — Two-phase flexible aggregation (sum+count, max+min, etc.)
4. filter — Keep only elements that satisfy a condition
5. distinct — Remove duplicate elements from an RDD
6. map / flatMap — Transform each element in an RDD
7. sortBy / sortByKey — Sort an RDD by value or key
8. Broadcast Variables — Efficiently share read-only data across all Executors

1.1 What is a Key-Value Pair RDD?

A key-value pair RDD (also called PairRDD) is simply an RDD where each element is a Python tuple of exactly TWO items: (key, value). The key identifies the group; the value is the data associated with it.

Creating Key-Value Pair RDDs

```
# Method 1: From a Python list of tuples
transactions = [('store1', 100), ('store2', 200), ('store1', 150)]
rdd = sc.parallelize(transactions)

# Method 2: Using map() to CREATE pairs from regular RDD
names = ['Raj-Mumbai', 'Priya-Delhi', 'Amit-Mumbai']
rdd = sc.parallelize(names)
pair_rdd = rdd.map(lambda x: (x.split('-')[1], x.split('-')[0]))
# Result: [('Mumbai','Raj'), ('Delhi','Priya'), ('Mumbai','Amit')]
```

```
# How to get SparkContext from SparkSession
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('LevelUp').getOrCreate()
sc = spark.sparkContext    # sc = SparkContext
```

Module 2: reduceByKey — Aggregate with Power

reduceByKey is the most commonly used RDD aggregation in all of PySpark. It combines all values for each key using a binary function — a function that takes TWO values and returns ONE. It is the engine behind word counts, sales totals, and every 'sum by group' operation.

> **Sateesh Says 💡**
>
> *Imagine Swiggy receives orders all day: ('Mumbai',350), ('Delhi',200), ('Mumbai',450), ('Delhi',300)...*
>
> *At end of day, the manager asks: 'What is total revenue from each city today?'*
>
> *reduceByKey(lambda x,y: x+y) does exactly that — sums up all values for each city in parallel across the cluster!*
>
> *This is the 'GROUP BY + SUM' of the RDD world. Master it and you understand distributed aggregation.*

2.1 How reduceByKey Works Internally

Understanding the internals is what separates senior engineers from beginners. reduceByKey does NOT simply group all values and then reduce — it PARTIALLY reduces within each partition FIRST, then shuffles, then reduces again. This is called combine-before-shuffle and it makes it dramatically more efficient than groupByKey.

reduceByKey Internal Execution — Step by Step

```
Input RDD: [('s1',100), ('s2',200), ('s1',150), ('s2',300), ('s1',200)]
Partitioned across 2 Executors:

Executor 1 — Partition 0:
[('s1',100), ('s2',200), ('s1',150)]

Executor 2 — Partition 1:
[('s2',300), ('s1',200)]

PHASE 1: LOCAL COMBINE (within each partition — NO shuffle yet)
Executor 1 locally combines:
s1: 100 + 150 = 250    s2: 200
Local result: [('s1',250), ('s2',200)]

Executor 2 locally combines:
s1: 200    s2: 300
Local result: [('s1',200), ('s2',300)]

PHASE 2: SHUFFLE (send same keys to same reducer)
All 's1' values -> Reducer A:  [250, 200]
All 's2' values -> Reducer B:  [200, 300]

PHASE 3: FINAL REDUCE
s1: 250 + 200 = 450   s2: 200 + 300 = 500
```

```
Output: [('s1', 450), ('s2', 500), ('s3', 120)]
```

2.2 Code Examples — From Class

Example 1: Store Sales Total

Store sales aggregation using reduceByKey

```
from pyspark import SparkContext
sc = spark.sparkContext

# Store sales data: (store_name, sale_amount)
transactions = [
    ('store1', 100), ('store2', 200),
    ('store1', 150), ('store2', 300),
    ('store3', 120), ('store1', 200)
]

# Step 1: Create the RDD
transactions_rdd = sc.parallelize(transactions)

# Step 2: Apply reduceByKey to sum sales per store
# lambda x, y: x + y  =>  takes two values, returns their sum
total_sales_rdd = transactions_rdd.reduceByKey(lambda x, y: x + y)

# Step 3: Collect and print results
total_sales = total_sales_rdd.collect()
for store, total in total_sales:
    print(f'{store}: {total}')
```

Output

```
store1: 450   (100 + 150 + 200)
store2: 500   (200 + 300)
store3: 120
```

Example 2: Find Minimum Score Per Student

reduceByKey with min — find lowest score per student

```
scores = [
    ('Alice', 85), ('Bob', 78),
    ('Alice', 92), ('Bob', 88),
    ('Charlie', 79)
]
scores_rdd = sc.parallelize(scores)

# Find MINIMUM score per student
# lambda x, y: min(x, y)  => returns whichever is smaller
min_scores_rdd = scores_rdd.reduceByKey(lambda x, y: min(x, y))

for student, score in min_scores_rdd.collect():
```

```
    print(f'{student}: {score}')

# Similarly — find MAXIMUM score:
max_scores_rdd = scores_rdd.reduceByKey(lambda x, y: max(x, y))

# Find TOTAL score (sum):
total_rdd = scores_rdd.reduceByKey(lambda x, y: x + y)

# Word count classic using reduceByKey:
words = ['apple','banana','apple','cherry','banana','apple']
word_pairs = sc.parallelize(words).map(lambda w: (w, 1))
word_counts = word_pairs.reduceByKey(lambda a, b: a + b)
# [('apple', 3), ('banana', 2), ('cherry', 1)]
```

Output: min scores

```
Alice:   85
Bob:     78
Charlie: 79
```

2.3 Key Rules — What Functions Work with reduceByKey

Critical Rule: The Function Must Be Associative AND Commutative

Associative: (a op b) op c == a op (b op c) [addition is associative]

Commutative: a op b == b op a [addition is commutative]

WHY: Spark processes partitions in any order, in parallel. The result must be the same regardless of order.

VALID: + (addition), * (multiplication), max(), min(), string concatenation with sort

INVALID: subtraction (-), division (/), string concatenation without sort (order matters!)

2.4 Interview Questions — reduceByKey

IQ-01 **What is reduceByKey in PySpark? How is it different from reduce()?**

reduceByKey operates on PairRDDs (key-value tuples). It groups values by key and applies a binary function to combine them — one result per key. reduce() operates on ALL elements regardless of key, returning a single value for the entire RDD. reduceByKey returns a new RDD; reduce() returns a scalar.

IQ-02 **Does reduceByKey trigger a shuffle? Explain.**

Yes, reduceByKey triggers a shuffle because all values for the same key must be on the same node to be combined. HOWEVER — unlike groupByKey — it partially reduces within each partition BEFORE shuffling, which significantly reduces network data transfer. This is the key optimization that makes reduceByKey preferred over groupByKey for aggregations.

IQ-03 Why is reduceByKey more efficient than groupByKey for aggregations?

reduceByKey applies the reduction function locally within each partition first (map-side combine), then shuffles only the partially reduced results. groupByKey collects ALL raw values for each key and sends them all across the network before any aggregation — much more network traffic and memory usage. For large datasets, this difference can be 10-100x in performance.

IQ-04 Can you use reduceByKey to find the maximum value per key? Show the lambda.

Yes: rdd.reduceByKey(lambda x, y: x if x > y else y) OR rdd.reduceByKey(lambda x, y: max(x, y)). The function takes two values and returns the larger one. Spark applies this repeatedly until one value per key remains.

Module 3: groupByKey — Group and Iterate

groupByKey groups all values for each key into an iterable (like a generator). Unlike reduceByKey, it does NOT reduce — it collects all values as a list so you can apply any arbitrary operation afterward. It is more flexible but more memory-intensive.

> **Sateesh Says** 💡
> *Think of a school principal asking: 'Give me ALL students in each class — names, marks, attendance — everything.'*
> *Not a summary, not a total — the ENTIRE list per class. That is groupByKey.*
> *When you need to do something complex with each group — like count unique pages per user, or list all orders per customer — groupByKey is the right tool.*
> *BUT: never use groupByKey just to then do a sum or count. Use reduceByKey for that! groupByKey loads everything into memory first.*

3.1 How groupByKey Works

groupByKey shuffles all values for each key to the same partition and returns a ResultIterable. You must call mapValues(list) to convert the iterable to a Python list for printing or further processing.

groupByKey Internal Flow

```
Input RDD: [('user1','page1'), ('user2','page1'), ('user1','page2')]

AFTER groupByKey():
('user1', ResultIterable([page1, page2]))
('user2', ResultIterable([page1]))
('user3', ResultIterable([page1]))

ResultIterable is a lazy generator — NOT a list!
You MUST call .mapValues(list) to convert it:
('user1', ['page1', 'page2'])   <- NOW it's a proper Python list
```

3.2 Code Examples — From Class

Example 1: Pages Visited Per User (Web Log Analysis)

groupByKey — web server log analysis

```
# Web server log: (user_id, page_visited)
data = [
    ('user1', 'page1'), ('user2', 'page1'),
    ('user1', 'page2'), ('user2', 'page2'),
```

```
    ('user3', 'page1')
]

rdd = sc.parallelize(data)

# Apply groupByKey — groups all pages for each user
grouped_rdd = rdd.groupByKey()

# IMPORTANT: mapValues(list) converts ResultIterable -> Python list
result = grouped_rdd.mapValues(list).collect()
print(result)

# Without mapValues(list) — you get a generator object (unreadable)
raw = rdd.groupByKey().collect()
print(raw)   # [('user1', <ResultIterable>), ...]  NOT useful!
```

Output with mapValues(list)

```
[('user1', ['page1', 'page2']), ('user2', ['page1', 'page2']), ('user3',
['page1'])]
```

Example 2: Sales Data Grouped by Product

groupByKey — product sales grouping

```
sales_data = [
    ('product1', 100), ('product2', 200),
    ('product1', 300), ('product3', 400),
    ('product2', 500)
]

rdd = sc.parallelize(sales_data)

# Group all sale amounts per product
grouped = rdd.groupByKey().mapValues(list).collect()
print(grouped)
# [('product1',[100,300]), ('product2',[200,500]), ('product3',[400])]

# Now you can do complex per-group operations:
# Calculate average per product
avg_per_product = rdd.groupByKey() \
    .mapValues(lambda vals: sum(vals) / len(list(vals))) \
    .collect()
# Note: vals is a ResultIterable — list() converts it once

# Count items per group
count_per_product = rdd.groupByKey() \
    .mapValues(lambda vals: len(list(vals))) \
    .collect()
```

Output

```
[('product1', [100, 300]), ('product2', [200, 500]), ('product3', [400])]
```

3.3 groupByKey vs reduceByKey — Critical Comparison

groupByKey	reduceByKey
Returns: (key, Iterable[values])	Returns: (key, aggregated_value)
ALL values shuffled across network	Partial combine BEFORE shuffle
Higher memory usage (full list)	Lower memory (only final value)
More flexible (any operation)	Must use a single binary function
Use when: need entire list per key	Use when: simple aggregate (sum/max/min)
Slow for large datasets	FAST — preferred for aggregations
rdd.groupByKey().mapValues(list)	rdd.reduceByKey(lambda x,y: x+y)

3.4 Interview Questions — groupByKey

IQ-05 **Why should you prefer reduceByKey over groupByKey for counting?**

groupByKey shuffles ALL values for each key across the network before any reduction. For a key with 10 million values, all 10 million are transferred. reduceByKey does a local partial count within each partition first, then shuffles much smaller intermediate results. In production, this can be the difference between a job completing in 5 minutes vs 2 hours — or even crashing with OOM (Out of Memory).

IQ-06 **What does mapValues(list) do after groupByKey?**

groupByKey returns a ResultIterable (a lazy Python generator object) as the value for each key. Calling mapValues(list) applies Python's list() constructor to each value — converting the generator to a concrete Python list. Without this, you cannot index, print, or iterate over the values normally. Always use mapValues(list) when you need to work with grouped results.

IQ-07 **When would you CHOOSE groupByKey over reduceByKey?**

When you need the complete list of values per key to perform a complex operation that cannot be expressed as a simple binary reduction. Examples: collecting unique page visits per user (need distinct), computing the median (need sorted list), building a list of all customer orders (need full list). If you just need sum/count/max/min, always use reduceByKey.

Module 4: aggregateByKey — The Two-Phase Aggregator

aggregateByKey is the MOST POWERFUL and MOST FLEXIBLE key-value aggregation in PySpark. It lets you compute aggregations that require maintaining multiple values per key — like both sum AND count simultaneously to compute averages. It is the expert-level alternative to both reduceByKey and groupByKey.

> **Sateesh Says 💡**
>
> *Imagine you are a teacher calculating the average test score of each student from 3 different tests.*
>
> *You cannot just 'reduce' scores because average is not a simple binary function (avg(a,b) != avg(a,b,c)/3).*
>
> *aggregateByKey says: in each partition, accumulate (sum, count) together. Then merge those accumulators across partitions.*
>
> *This is exactly how data pipelines at HDFC Bank compute average transaction amount per customer at massive scale!*

4.1 The Three Parameters — Explained Simply

aggregateByKey takes exactly THREE arguments: zeroValue, seqOp, and combOp. Understanding each one is the key to mastering this function.

aggregateByKey Syntax

```
rdd.aggregateByKey(zeroValue, seqOp, combOp)

zeroValue  = The initial/neutral accumulator value for each key.
             Like the starting score (0,0) before reading any data.

seqOp      = Sequential operation. HOW to combine one value from
             the RDD with the current accumulator WITHIN a partition.
             signature: (accumulator, new_value) -> new_accumulator

combOp     = Combine operation. HOW to merge two accumulators from
             DIFFERENT partitions into one final accumulator.
             signature: (acc1, acc2) -> merged_acc
```

4.2 Deep Dive: Average Score Per Student

This is the canonical example from class. We have student scores distributed across partitions and want to compute the average score per student.

Class example: aggregateByKey to compute average

```
data = [
    ('Alice', 85),  # Alice test 1
    ('Bob',   70),  # Bob test 1
    ('Alice', 92),  # Alice test 2
    ('Bob',   88),  # Bob test 2
    ('Alice', 78),  # Alice test 3
    ('Bob',   95)   # Bob test 3
]

rdd = spark.sparkContext.parallelize(data, 3)   # 3 partitions

# zeroValue: (sum=0, count=0) — start from zero for every key
zero_value = (0, 0)

# seqOp: add one score to running (sum, count)
# acc = current accumulator (sum_so_far, count_so_far)
# value = the new score to add
seqOp = lambda acc, value: (acc[0] + value, acc[1] + 1)

# combOp: merge two (sum, count) tuples from different partitions
combOp = lambda acc1, acc2: (acc1[0] + acc2[0], acc1[1] + acc2[1])

result = rdd.aggregateByKey(zero_value, seqOp, combOp).collect()
print(result)

# Compute the actual average:
averages = rdd.aggregateByKey(zero_value, seqOp, combOp) \
    .mapValues(lambda x: x[0] / x[1]) \  # sum / count = average
    .collect()
print(averages)
```

Output

```
result:   [('Alice', (255, 3)), ('Bob', (253, 3))]
averages: [('Alice', 85.0), ('Bob', 84.33...)]
```

4.3 Step-by-Step Walkthrough — Exactly as Taught in Class

This is the detailed trace from class showing exactly what happens in each partition:

Partition-by-partition trace (from Sateesh's class notes)

```
rdd = spark.sparkContext.parallelize(data, 3)
# Spark distributes 6 items across 3 partitions:

PARTITION 0: [('Alice',85), ('Bob',70), ('Alice',92), ('Bob',88)]
PARTITION 1: [('Alice',78), ('Bob',95)]

——— PHASE 1: seqOp runs WITHIN each partition ———

Partition 0 — seqOp:
Alice: start=(0,0), see 85 -> (0+85, 0+1)=(85,1)
Alice: see 92        -> (85+92, 1+1)  = (177, 2)
Bob:   start=(0,0), see 70 -> (0+70, 0+1)=(70,1)
Bob:   see 88        -> (70+88, 1+1)  = (158, 2)
Partition 0 result: Alice=(177,2), Bob=(158,2)
```

```
Partition 1 — seqOp:
Alice: start=(0,0), see 78 -> (78, 1)
Bob:   start=(0,0), see 95 -> (95, 1)
Partition 1 result: Alice=(78,1), Bob=(95,1)

——— PHASE 2: combOp merges results across partitions ———

Alice: combOp((177,2), (78,1))  = (177+78, 2+1) = (255, 3)
Bob:   combOp((158,2), (95,1))  = (158+95, 2+1) = (253, 3)

——— FINAL: mapValues to compute average ———
Alice: 255/3 = 85.0
Bob:   253/3 = 84.33
```

4.4 More Examples with aggregateByKey

Example 2: Sum and Count (user scores)

aggregateByKey — user scores from class

```
data = [('user1',10), ('user2',20), ('user1',30), ('user2',40), ('user3',50)]
rdd = spark.sparkContext.parallelize(data)

zero_value = (0, 0)   # (sum, count)
seqOp  = lambda acc, value: (acc[0] + value,   acc[1] + 1)
combOp = lambda acc1, acc2: (acc1[0] + acc2[0], acc1[1] + acc2[1])

result = rdd.aggregateByKey(zero_value, seqOp, combOp).collect()
print(result)
# [('user1', (40, 2)), ('user2', (60, 2)), ('user3', (50, 1))]
# user1: sum=40 (10+30), count=2
# user2: sum=60 (20+40), count=2
# user3: sum=50,         count=1
```

Example 3: Min and Max per key

aggregateByKey — find both min and max simultaneously

```
# Find (min, max) temperature per city
temps = [('Delhi',35),('Mumbai',32),('Delhi',28),('Mumbai',38),('Delhi',40)]
rdd = sc.parallelize(temps)

# zeroValue: (min=+infinity, max=-infinity)
zero = (float('inf'), float('-inf'))

# seqOp: update min and max with new temperature
seqOp  = lambda acc, v: (min(acc[0], v), max(acc[1], v))

# combOp: merge two (min,max) pairs from different partitions
combOp = lambda a1, a2: (min(a1[0],a2[0]), max(a1[1],a2[1]))

result = rdd.aggregateByKey(zero, seqOp, combOp).collect()
```

```
print(result)
# [('Delhi', (28, 40)), ('Mumbai', (32, 38))]
```

4.5 Interview Questions — aggregateByKey

IQ-08 **Explain the three parameters of aggregateByKey.**

zeroValue: the initial accumulator value for each key (like starting score board = (0,0)). seqOp: how to combine a new RDD value with the current accumulator WITHIN a partition — signature is (accumulator, value) -> new_accumulator. combOp: how to merge two accumulators from DIFFERENT partitions — signature is (acc1, acc2) -> merged_acc. zeroValue is applied once per key per partition before processing begins.

IQ-09 **Why can't you use reduceByKey to compute the average per key?**

reduceByKey needs a function f(a,b)->c where the result type is the SAME as the input type. Average requires tracking two values (sum, count) which is a tuple — a different type than the individual scores (integer). aggregateByKey allows the accumulator type to DIFFER from the RDD value type, making it possible to accumulate (sum, count) tuples from individual integer values.

IQ-10 **What is the difference between seqOp and combOp in aggregateByKey?**

seqOp runs WITHIN a partition — it combines individual RDD values with the accumulator. It runs on every element. combOp runs ACROSS partitions — it merges two accumulators that came from different partitions. combOp never sees raw RDD values; it only ever receives accumulators. This two-phase design allows efficient parallel computation before and after the shuffle.

Module 5: filter() and distinct() — Clean Your Data

filter() and distinct() are fundamental data cleaning operations. filter() selects elements matching a condition. distinct() removes duplicate elements. Both are used constantly in production ETL pipelines.

5.1 filter() — Keep What You Need

Sateesh Says 💡

Think of filter() like a security guard at an Infosys gate: only badge-holders get in, everyone else is rejected.

You define the rule (condition), and filter() applies it to every element in the RDD in parallel.

Use filter() to remove nulls, keep only valid records, select records above a threshold, or clean noisy data.

Every single real-world data pipeline starts with filtering bad data. It's your first line of defense!

filter() — Syntax and all patterns

```
# Basic syntax:
filtered_rdd = rdd.filter(lambda element: condition)
# condition must return True (keep) or False (discard)

# Pattern 1: Filter even numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rdd = sc.parallelize(numbers)
even_rdd = rdd.filter(lambda x: x % 2 == 0)
print(even_rdd.collect())   # [2, 4, 6, 8, 10]

# Pattern 2: Filter by value threshold (from sensor use case)
sensor_data = [('M001',85), ('M002',95), ('M001',75), ('M002',105)]
rdd = sc.parallelize(sensor_data)
abnormal = rdd.filter(lambda x: x[1] > 90)   # reading above 90
print(abnormal.collect())  # [('M002',95), ('M002',105)]

# Pattern 3: Filter non-null values
data = [('Raj', 85000), (None, 70000), ('Priya', None), ('Amit', 78000)]
rdd = sc.parallelize(data)
clean = rdd.filter(lambda x: x[0] is not None and x[1] is not None)
print(clean.collect())  # [('Raj',85000), ('Amit',78000)]

# Pattern 4: Filter strings (startswith, contains, length)
cities = ['Mumbai', 'Delhi', 'Mangalore', 'Mysuru', 'Madurai']
rdd = sc.parallelize(cities)
m_cities = rdd.filter(lambda x: x.startswith('M'))
print(m_cities.collect())  # ['Mumbai','Mangalore','Mysuru','Madurai']

# Pattern 5: Filter key-value pairs
sales = [('laptop',50000),('phone',20000),('tv',80000),('pen',50)]
rdd = sc.parallelize(sales)
high_value = rdd.filter(lambda kv: kv[1] > 25000)
print(high_value.collect())  # [('laptop',50000), ('tv',80000)]
```

```
# Pattern 6: Using a named function (cleaner for complex conditions)
def is_valid_record(record):
    emp_id, name, salary = record
    return emp_id is not None and salary > 0 and len(name) > 0
valid_records = employee_rdd.filter(is_valid_record)
```

5.2 distinct() — Remove Duplicates

Sateesh Says 💡

Imagine Zomato's database has the same order logged 3 times due to a network retry. distinct() removes those duplicates.

distinct() compares entire elements — every field must match for two elements to be considered duplicates.

For key-value pairs, BOTH the key AND value must be identical for deduplication. If you want distinct keys only, use keys().distinct().

Always profile your data for duplicates before groupBy or join. Duplicates cause inflated counts — the #1 reporting bug in production!

distinct() — All patterns

```
# Basic distinct on simple values
numbers = [1, 2, 3, 4, 5, 1, 2]
rdd = sc.parallelize(numbers)
unique = rdd.distinct()
print(unique.collect())   # [1, 2, 3, 4, 5]

# distinct on tuples — BOTH key and value must match
logs = [('U001','/home'), ('U001','/about'), ('U001','/home')]
rdd = sc.parallelize(logs)
unique_logs = rdd.distinct()
print(unique_logs.collect())  # [('U001','/home'), ('U001','/about')]
# Note: ('U001','/home') appeared TWICE -> removed to once

# Get distinct KEYS only
sales = [('product1',100),('product2',200),('product1',300)]
rdd = sc.parallelize(sales)
distinct_products = rdd.keys().distinct()
print(distinct_products.collect())  # ['product1', 'product2']

# Count unique elements
unique_count = rdd.distinct().count()

# distinct().count() is a common pattern — count unique values
cities = [('Delhi',1),('Mumbai',2),('Delhi',3),('Chennai',4)]
unique_city_count = sc.parallelize(cities).keys().distinct().count()
print(unique_city_count)   # 3
```

Output

```
numbers distinct: [1, 2, 3, 4, 5]
logs distinct:    [('U001', '/home'), ('U001', '/about')]
distinct products: ['product1', 'product2']
```

5.3 Interview Questions — filter and distinct

IQ-11 **Is filter() a narrow or wide transformation? Does it cause a shuffle?**

filter() is a NARROW transformation — it evaluates each element independently, so each output partition depends only on one input partition. No shuffle occurs. This makes filter() extremely fast and should always be applied as early as possible in your pipeline ('filter early, filter often') to reduce data volume before any shuffling operations like groupBy or join.

IQ-12 **What is the performance cost of distinct()?**

distinct() triggers a SHUFFLE because it needs to identify and eliminate duplicates across all partitions — elements with the same value may be on different nodes. Internally it is implemented as map(x -> (x, null)).reduceByKey(...).keys(). For large datasets, distinct() can be expensive. Always ask: do I really need global distinct? Sometimes a per-partition distinct is sufficient and much cheaper.

IQ-13 **How would you filter and then count abnormal sensor readings in one pipeline?**

sensor_data_rdd.filter(lambda x: x[1] > threshold).count() — chain filter() directly to count(). This is lazy: Spark builds one DAG with filter+count and executes it in a single pass. You do not need to store the filtered RDD separately unless you need to reuse it. If reusing, persist() after filter() to avoid re-filtering on every action.

Module 6: map(), flatMap(), sortBy() and sortByKey()

These are the transformation workhorse operations of PySpark RDDs. map() transforms each element. flatMap() transforms and flattens. sortBy() sorts by any custom key. sortByKey() sorts by the pair's key. Every RDD pipeline uses these.

6.1 map() — Transform Every Element

Sateesh Says 💡

map() is like a production conveyor belt at an Infosys factory. Each item goes in, gets processed, comes out transformed.

Every element in the RDD goes through your function. One input, one output. Always 1-to-1.

map() is a narrow transformation — no shuffle, runs in parallel, extremely fast.

Use map() to extract fields, create key-value pairs, apply formulas, clean strings, convert types.

map() — All important patterns

```
# Pattern 1: Create key-value pairs from raw data
transactions = [('C001','Item1',100), ('C002','Item2',150)]
rdd = sc.parallelize(transactions)
# Extract (customer_id, amount) pairs for reduceByKey
kv_rdd = rdd.map(lambda x: (x[0], x[2]))
# [('C001', 100), ('C002', 150)]

# Pattern 2: Apply a formula / computation
prices = [100, 200, 150, 300]
rdd = sc.parallelize(prices)
tax_rdd = rdd.map(lambda p: p * 1.18)   # Add 18% GST
# [118.0, 236.0, 177.0, 354.0]

# Pattern 3: Extract specific field from tuple
sensor_data = [('M001',85,'2024-08-25'), ('M002',95,'2024-08-25')]
rdd = sc.parallelize(sensor_data)
machine_reading = rdd.map(lambda x: (x[0], x[1]))
# [('M001', 85), ('M002', 95)]

# Pattern 4: String transformation
names = ['  raj  ', ' PRIYA ', 'amit ']
rdd = sc.parallelize(names)
clean = rdd.map(lambda s: s.strip().title())
# ['Raj', 'Priya', 'Amit']

# Pattern 5: Parse log lines
logs = ['2024-01-01 ERROR failed to connect',
        '2024-01-02 INFO server started']
rdd = sc.parallelize(logs)
parsed = rdd.map(lambda line: {
    'date':  line.split()[0],
    'level': line.split()[1],
    'msg':   ' '.join(line.split()[2:])
})
```

6.2 flatMap() — Transform and Flatten

flatMap() is like map() but each element can produce ZERO, ONE, or MANY output elements. The result is a flat RDD — not a list of lists. It is essential for text processing, log parsing, and exploding arrays.

flatMap() — The word-splitting powerhouse

```
# The key difference: map() vs flatMap()
sentences = ['I love PySpark', 'Spark is fast']
rdd = sc.parallelize(sentences)

# map() — one output per input (a list of lists)
mapped = rdd.map(lambda s: s.split()).collect()
print(mapped)
# [['I','love','PySpark'], ['Spark','is','fast']]  <- nested!

# flatMap() — flattens the output (a single flat list of words)
flat = rdd.flatMap(lambda s: s.split()).collect()
print(flat)
# ['I', 'love', 'PySpark', 'Spark', 'is', 'fast']  <- flat!

# Word count pipeline using flatMap (Assignment pattern):
feedback = [
    'The flight was delayed and service was terrible',
    'The food was good but seats were uncomfortable'
]
rdd = sc.parallelize(feedback)
word_counts = rdd \
    .flatMap(lambda line: line.lower().split()) \   # split words
    .map(lambda word: (word, 1)) \                  # make pairs
    .reduceByKey(lambda a, b: a + b) \             # count words
    .sortBy(lambda x: x[1], ascending=False)       # sort by count
word_counts.collect()
```

6.3 sortBy() and sortByKey()

Sateesh Says 💡

sortBy() lets you sort by ANY computed value — length of list, custom score, field 2 of a tuple.
sortByKey() sorts key-value pair RDDs by their KEY. The key must be naturally comparable (string, int).
Both are WIDE transformations — they trigger a shuffle to globally order the data across all partitions.
Use take(N) after sorting instead of collect() to get top-N without loading everything to the Driver!

sortBy() and sortByKey() — All patterns

```
# sortByKey() — sort key-value RDD by key (alphabetical/numeric)
sales = [('C003',350),('C001',300),('C002',450)]
rdd = sc.parallelize(sales)

asc_rdd  = rdd.sortByKey()              # ascending (A-Z, 0-9)
```

```
desc_rdd = rdd.sortByKey(ascending=False) # descending (Z-A, 9-0)
print(asc_rdd.collect())
# [('C001',300), ('C002',450), ('C003',350)]

# sortBy() — sort by any computed key (the lambda determines order)
# Sort by VALUE (second element):
sorted_by_val = rdd.sortBy(lambda kv: kv[1], ascending=False)
print(sorted_by_val.collect())
# [('C002',450), ('C003',350), ('C001',300)]

# From Use Case 1: sort by LENGTH of list (most active users first)
grouped_rdd = ...  # [('U001',['pg1','pg2']), ('U002',['pg1'])]
sorted_rdd = grouped_rdd.sortBy(lambda x: len(x[1]), ascending=False)
# Users with MOST pages come first

# From Use Case 2: sort by count of abnormal readings
sorted_abnormal = total_counts.sortBy(lambda x: x[1], ascending=False)

# take(N) — get top N after sorting WITHOUT collecting everything!
top_2 = sorted_rdd.take(2)   # returns first 2 as Python list
top_1 = sorted_rdd.first()   # returns single first element
```

6.4 Interview Questions — map, flatMap, sortBy

IQ-14 What is the difference between map() and flatMap()?

map() is a 1-to-1 transformation: each input element produces exactly ONE output element. If your function returns a list, the RDD contains a list as one element. flatMap() is a 1-to-N transformation: each input element can produce ZERO, ONE, or MANY output elements, and the results are flattened into a single flat RDD. Use flatMap() for splitting sentences into words, exploding arrays, or parsing structured logs.

IQ-15 What is the difference between sortBy() and sortByKey()?

sortByKey() works only on PairRDDs (key-value tuples) and sorts by the first element (the key). It is a shorthand for sortBy(lambda x: x[0]). sortBy() is more flexible — you provide a custom lambda to determine the sort key, which can be the second element, the length of a nested list, or any computed value. Both trigger a shuffle. Both accept ascending=True/False.

IQ-16 Why should you use take(N) instead of collect() after sorting?

After sortBy(), calling collect() brings ALL sorted records to the Driver — potentially gigabytes of data. If you only need the top 10 results, take(10) brings only those 10 rows. Spark is smart about this: with take(N) it uses a partial sort that stops as soon as it has enough results, avoiding the full sort in many cases. In production: always use take(N) for top-N queries, never collect() the entire sorted RDD.

Module 7: Broadcast Variables — Efficient Shared Data

Broadcast variables allow you to share a read-only dataset efficiently with ALL Executors in the cluster. Without broadcast, Spark sends a copy of the variable with every single Task. With broadcast, it sends ONE copy per Executor (not per Task). For frequently used lookup tables, this can save gigabytes of network traffic.

> **Sateesh Says 💡**
>
> *Imagine you are distributing question papers to 1000 students at IIT Hyderabad.*
>
> *WITHOUT broadcast: the exam coordinator walks to every single student desk 1000 times with individual paper copies. 1000 trips!*
>
> *WITH broadcast: one photocopier makes 10 copies for 10 rooms, each room distributes internally. 10 trips!*
>
> *In Spark: WITHOUT broadcast — every Task gets its own copy of the stopwords list = 1000s of copies sent over network.*
>
> *WITH broadcast — ONE copy sent per Executor = 10 copies for 10 machines. Massive savings!*

7.1 How Broadcast Works — Internals

Broadcast variable internals

```
# WITHOUT broadcast (bad for large shared data):
stopwords = ['the', 'and', 'was', 'but', 'on', 'this']
# Spark serializes stopwords and sends it with EVERY task
# If you have 200 tasks, stopwords is sent 200 times!
filtered = rdd.filter(lambda word: word not in stopwords)

# WITH broadcast (efficient):
stopwords = ['the', 'and', 'was', 'but', 'on', 'this']
stopwords_bc = sc.broadcast(stopwords)   # creates broadcast var
# Spark sends ONE copy to each Executor (not each Task!)
# Access the value inside tasks using .value
filtered = rdd.filter(lambda word: word not in stopwords_bc.value)
# .value retrieves the actual Python object on each Executor

# Important methods:
bc = sc.broadcast(my_data)     # create broadcast variable
bc.value                        # access the data inside tasks
bc.unpersist()                  # remove from Executor memory
bc.destroy()                    # remove from ALL memory (permanent)
```

7.2 Assignment Example — Broadcast Stopwords

From class: Word count on airline feedback with broadcast stopwords. This is the full solution:

Complete assignment solution — word count with broadcast

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('WordCount').getOrCreate()
sc = spark.sparkContext

# Sample airline feedback data
feedback_data = [
    'The flight was delayed and the customer service was terrible.',
    'The food was good but the seats were uncomfortable.',
    'The flight was smooth and the crew was friendly.',
    'I had a great experience on this flight.',
    'The entertainment system was not working properly.'
]

rdd = sc.parallelize(feedback_data)

# Define stopwords
stopwords = ['the','and','was','but','on','this','had','a','in','it','i']

# Broadcast stopwords to ALL Executors ONCE
stopwords_broadcast = sc.broadcast(stopwords)

# Pipeline:
# 1. flatMap: split each sentence into words, convert to lowercase
# 2. filter:  remove stopwords using broadcast variable
# 3. map:     create (word, 1) pairs
# 4. reduceByKey: count occurrences of each word
# 5. sortBy:  sort by count descending (most frequent first)

result = rdd \
    .flatMap(lambda line: line.lower().split()) \
    .filter(lambda word: word not in stopwords_broadcast.value) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b) \
    .sortBy(lambda x: x[1], ascending=False)

for word, count in result.collect():
    print(f'{word}: {count}')
```

Output (top words)

```
flight:    3
customer:  1
service:   1
delayed:   1
terrible:  1
food:      1
good:      1
... (continues)
```

Pro Tip

Use broadcast variables for: lookup tables, small reference datasets, ML model parameters, configuration dictionaries.

Rule of thumb: broadcast anything smaller than 100MB that is accessed by many tasks.

Large broadcast variables (>200MB) can actually SLOW things down due to serialization overhead — test both ways.

Module 8: Use Cases — Real-World RDD Pipelines

These three use cases are exactly what Sateesh taught in class. Each use case applies multiple operations in a real pipeline. Study these — they are the template for 90% of RDD coding questions in interviews.

Sateesh Says 💡

In EVERY PySpark interview, they give you a dataset and ask you to chain 4-5 operations together.

These 3 use cases cover the exact patterns that Amazon, Walmart, TCS Digital, and Databricks test.

After each solution, I will show you what each step does and WHY it is there — not just the code!

01 USE CASE

Website Log Data Analysis

Objective: Identify the top 2 users with the most unique page visits

Dataset — from class

```
logs = [
    ('U001', '/home',     '2024-08-25 10:00:00'),
    ('U002', '/about',    '2024-08-25 10:05:00'),
    ('U001', '/products', '2024-08-25 10:10:00'),
    ('U002', '/home',     '2024-08-25 10:15:00'),
    ('U003', '/products', '2024-08-25 10:20:00'),
    ('U001', '/home',     '2024-08-25 10:25:00')   # duplicate!
]
# Note: U001 visited /home TWICE — that's only 1 unique page!
```

Expected Output

```
[('U001', ['/home', '/products']), ('U002', ['/about', '/home'])]
# U001: 2 unique pages (/home + /products — the duplicate removed)
# U002: 2 unique pages
# U003: 1 unique page (excluded from top 2)
```

Operations to Perform: map -> distinct -> groupByKey -> mapValues -> sortBy -> take

STEP 1 — **map — Extract (user_id, page) pairs**

Remove the timestamp column. We only need user and page.

STEP 2 — **distinct — Remove duplicate visits**

U001 visited /home twice. Count it as ONE unique visit.

STEP 3

groupByKey + mapValues(list) — Group pages per user

Collect all unique pages for each user into a list.

STEP 4

sortBy — Sort by number of unique pages (descending)

User with MOST unique pages comes first.

STEP 5

take(2) — Return only the top 2 users

Do NOT use collect() — we only want 2 results.

Complete Solution with Explanations

```
rdd = spark.sparkContext.parallelize(logs)

# STEP 1: map — keep only (user_id, page), drop timestamp
# x[0]=user_id, x[1]=page, x[2]=timestamp (we discard x[2])
user_pages_rdd = rdd.map(lambda x: (x[0], x[1]))
# [('U001','/home'),('U002','/about'),('U001','/products'),
#  ('U002','/home'),('U003','/products'),('U001','/home')]

# STEP 2: distinct — remove ('U001','/home') duplicate
distinct_rdd = user_pages_rdd.distinct()
# [('U001','/home'),('U002','/about'),('U001','/products'),
#  ('U002','/home'),('U003','/products')]   <- /home duplicate gone!

# STEP 3: groupByKey + mapValues(list) — collect pages per user
grouped_rdd = distinct_rdd.groupByKey().mapValues(list)
# [('U001',['/home','/products']), ('U002',['/about','/home']),
#  ('U003',['/products'])]

# STEP 4: sortBy — users with MOST unique pages come first
# len(x[1]) = number of unique pages for user x[0]
sorted_rdd = grouped_rdd.sortBy(lambda x: len(x[1]), ascending=False)
# [('U001', 2 pages), ('U002', 2 pages), ('U003', 1 page)]

# STEP 5: take(2) — get top 2 users only
top_active_users = sorted_rdd.take(2)
print(top_active_users)
```

Final Output

```
[('U001', ['/home', '/products']), ('U002', ['/about', '/home'])]
```

02 USE CASE — Industrial Sensor Anomaly Detection

Objective: Identify machines with more than one abnormal reading (reading > 90) and rank them

Dataset — from class

```
sensor_data = [
    ('M001', 85,  '2024-08-25 10:00:00'),  # normal
    ('M002', 95,  '2024-08-25 10:05:00'),  # ABNORMAL
    ('M001', 75,  '2024-08-25 10:10:00'),  # normal
    ('M002', 105, '2024-08-25 10:15:00'),  # ABNORMAL
```

```
    ('M003', 65,  '2024-08-25 10:20:00'),  # normal
    ('M001', 95,  '2024-08-25 10:25:00'),  # ABNORMAL
    ('M002', 100, '2024-08-25 10:30:00'),  # ABNORMAL
]
```

Expected Output

```
[('M002', 3), ('M001', 1)]
# M002 has 3 abnormal readings (95, 105, 100) — ranked first
# M001 has 1 abnormal reading (95)
# M003 has 0 abnormal readings — excluded
```

Operations: filter -> map -> reduceByKey -> sortBy

STEP 1 — **filter — Keep only abnormal readings (> 90)**
M001@85, M001@75, M003@65 are normal — discard them.

STEP 2 — **map — Convert to (machine_id, 1) pairs**
We need to COUNT abnormal readings. Each one becomes a '1'.

STEP 3 — **reduceByKey — Sum the 1s per machine**
M002 has three 1s -> sum = 3. M001 has one 1 -> sum = 1.

STEP 4 — **sortBy — Sort by count descending**
Machine with MOST abnormal readings comes first.

Complete Solution

```
rdd = spark.sparkContext.parallelize(sensor_data)

# STEP 1: filter — only readings ABOVE 90 are abnormal
# x[1] is the sensor reading value
rdd2 = rdd.filter(lambda x: x[1] > 90)
# [('M002',95,...),('M002',105,...),('M001',95,...),('M002',100,...)]

# STEP 2: map — create (machine_id, 1) counting pairs
# x[0]=machine_id, we assign value=1 to count this occurrence
rdd3 = rdd2.map(lambda x: (x[0], 1))
# [('M002',1), ('M002',1), ('M001',1), ('M002',1)]

# STEP 3: reduceByKey — sum up all 1s per machine
rdd4 = rdd3.reduceByKey(lambda x, y: x + y)
# [('M002', 3), ('M001', 1)]

# STEP 4: sortBy — machine with MOST abnormal readings first
rdd5 = rdd4.sortBy(lambda x: x[1], ascending=False)
result = rdd5.collect()
print(result)
```

Final Output

```
[('M002', 3), ('M001', 1)]
```

03
USE CASE

E-Commerce Transaction Analysis

Objective: Calculate total amount spent by each customer and sort by customer ID

Dataset — from class

```
transactions = [
    ('C001', 'Item1', 100),
    ('C002', 'Item2', 150),
    ('C001', 'Item3', 200),
    ('C002', 'Item1', 300),
    ('C003', 'Item2', 250),
    ('C003', 'Item3', 100)
]
```

Expected Output

```
[('C001', 300), ('C002', 450), ('C003', 350)]
# C001: 100 + 200 = 300  (sorted first by ID)
# C002: 150 + 300 = 450
# C003: 250 + 100 = 350  (sorted last by ID)
```

Operations: map -> reduceByKey -> sortByKey

STEP 1

map — Extract (customer_id, amount) pairs

x[0]=customer_id, x[2]=amount. Discard x[1] (item name) — not needed for totals.

STEP 2

reduceByKey — Sum all amounts per customer

All purchases by same customer are added together.

STEP 3

sortByKey — Sort by customer ID alphabetically

C001 < C002 < C003 lexicographically (default sort order).

Complete Solution

```
rdd = spark.sparkContext.parallelize(transactions)

# STEP 1: map – extract (customer_id, amount), discard item name
# x[0]=C001, x[1]=Item1 (discarded), x[2]=100
rdd2 = rdd.map(lambda x: (x[0], x[2]))
# [('C001',100),('C002',150),('C001',200),('C002',300),
#  ('C003',250),('C003',100)]

# STEP 2: reduceByKey – sum all amounts per customer
rdd3 = rdd2.reduceByKey(lambda x, y: x + y)
# [('C001',300), ('C002',450), ('C003',350)]

# STEP 3: sortByKey – sort by customer ID alphabetically (C001<C002<C003)
rdd4 = rdd3.sortByKey()   # ascending=True by default
result = rdd4.collect()
print(result)
```

Final Output

```
[('C001', 300), ('C002', 450), ('C003', 350)]
```

Module 9: Assignment — Airline Feedback Word Count

This is the class assignment from the notebook. Build a full word count pipeline on airline customer feedback, using broadcast variables for stopwords, then sort results descending.

Sateesh Says 💡

Word count is the 'Hello World' of Big Data. But this version is production-grade — broadcast stopwords, case normalization, sorted output.

This exact pattern (flatMap -> filter -> map -> reduceByKey -> sortBy) is the blueprint for text analytics at any scale.

Zomato's sentiment analysis, Flipkart's product review mining, airline feedback systems — all use this pattern at terabyte scale!

9.1 Assignment Tasks

Task 1	Word Count: Count how many times each word appears in the dataset. Case-insensitive ('Flight' = 'flight'). Split on spaces.	*Hint: flatMap to split, map to create (word,1) pairs, reduceByKey to sum.*
Task 2	Sort the words by their count in DESCENDING order (most frequent first).	*Hint: sortBy(lambda x: x[1], ascending=False)*
Task 3	Apply capitalization to each word in the final output (first letter uppercase).	*Hint: mapValues or map to apply .capitalize() on each word.*
Task 4	Save the final sorted word counts to a file. Display the top 20 most frequent words.	*Hint: saveAsTextFile() for saving, take(20) for display.*

9.2 Full Assignment Solution

Complete solution — all 4 tasks

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('WordCount').getOrCreate()
sc = spark.sparkContext

# Dataset (in actual assignment – load from file)
feedback_data = [
    'The flight was delayed and the customer service was terrible.',
```

```
    'The food was good but the seats were uncomfortable.',
    'The flight was smooth and the crew was friendly.',
    'I had a great experience on this flight.',
    'The entertainment system was not working properly.'
]

rdd = sc.parallelize(feedback_data)

# STOPWORDS — broadcast to all Executors for efficiency
stopwords = ['the','and','was','but','on','this','had','a','in','it','i']
stopwords_bc = sc.broadcast(stopwords)

# TASK 1: Word Count — case insensitive, excluding stopwords
word_counts = rdd \
    .flatMap(lambda line: line.lower().split()) \  # Task1: lowercase+split
    .map(lambda word: word.strip('.,!?;:'))  \     # remove punctuation
    .filter(lambda word: word not in stopwords_bc.value) \ # no stopwords
    .filter(lambda word: len(word) > 0) \           # no empty strings
    .map(lambda word: (word, 1)) \                  # create (word,1) pairs
    .reduceByKey(lambda a, b: a + b)                # count occurrences

# TASK 2: Sort by count DESCENDING
sorted_counts = word_counts.sortBy(lambda x: x[1], ascending=False)

# TASK 3: Capitalize each word in output
capitalized = sorted_counts.map(lambda kv: (kv[0].capitalize(), kv[1]))

# TASK 4a: Display top 20
print('Top 20 Most Frequent Words:')
for word, count in capitalized.take(20):
    print(f'{word}: {count}')

# TASK 4b: Save to file
capitalized.saveAsTextFile('/output/airline_word_count/')
# For production: use Parquet or Delta — not text files!
# capitalized.toDF(['word','count']).write.parquet('/output/wc/')
```

Sample Output

```
Top 20 Most Frequent Words:
Flight:         3
Delayed:        1
Customer:       1
Service:        1
Terrible:       1
Food:           1
Good:           1
Seats:          1
Uncomfortable:  1
Smooth:         1
Crew:           1
Friendly:       1
Great:          1
Experience:     1
Entertainment:  1
System:         1
Not:            1
Working:        1
Properly:       1
```

Module 10: Master Comparison & Interview Cheat Sheet

10.1 All 5 Key-Value Operations — Side-by-Side

Operation	Output Type	Shuffle?	Memory	Best For
reduceByKey	(K, single_value)	Yes (optimised)	Low — partial agg first	sum, max, min, count
groupByKey	(K, Iterable)	Yes (heavy)	HIGH — full list in RAM	collect all values per key
aggregateByKey	(K, custom_acc)	Yes (optimised)	Medium — custom acc	avg, complex multi-value agg
filter	Same RDD type	No (narrow)	Same	remove unwanted records
distinct	Same RDD type	Yes (dedup shuffle)	Medium	remove exact duplicates
map	One elem per elem	No (narrow)	Same	transform / extract fields
flatMap	N elems per elem	No (narrow)	Same	split text, explode arrays
sortBy	Sorted RDD	Yes (global sort)	High	order results by any key
sortByKey	Sorted PairRDD	Yes (global sort)	High	order by key alphabetically

10.2 Top 10 Final Interview Questions

IQ-17 **What is the difference between reduceByKey and groupByKey? When would you use each?**

reduceByKey combines values per key using a binary function and does a partial combine BEFORE the shuffle — efficient for aggregations (sum, max, count). groupByKey sends ALL raw values to one reducer before any processing — more memory and network traffic but returns full value list per key. Use reduceByKey for any aggregation. Use groupByKey only when you need the complete list of values per key for complex per-group processing.

IQ-18 **What does sc.broadcast() do? Why is it important for performance?**

sc.broadcast() sends a read-only copy of a Python object to EVERY Executor node (once per node, not once per Task). Without it, Spark serializes the variable and ships it with every Task — potentially thousands of copies. With broadcast, only one copy per Executor is needed. Critical for lookup tables, stopword lists, and configuration objects used inside map/filter/flatMap lambdas.

IQ-19 How would you implement word count in PySpark RDD? Walk through each step.

1. sc.parallelize(text_data) or sc.textFile('path'). 2. flatMap(lambda line: line.lower().split()) — split each line into words. 3. map(lambda word: (word,1)) — create (word,1) pairs. 4. reduceByKey(lambda a,b: a+b) — count each word. 5. sortBy(lambda x: x[1], ascending=False) — sort by frequency. This is the canonical Big Data pipeline.

IQ-20 In Use Case 2 (sensor anomaly), why filter() BEFORE map() and not after?

Filter early to reduce data volume. If we map() first (create (machine,1) pairs) and then filter, we apply the map transformation to ALL records — including normal ones. By filtering first, we discard normal readings immediately and only process the subset of abnormal readings through map and reduceByKey. This reduces the number of elements processed by every subsequent operation. Always push filters as early in the pipeline as possible.

IQ-21 Why was distinct() necessary in Use Case 1 before groupByKey?

Without distinct(), ('U001','/home') appeared TWICE in the data (two visits at different timestamps). After groupByKey without distinct, U001's page list would be ['/home', '/products', '/home'] — with /home counted twice. The objective was UNIQUE page visits, so distinct() on the (user,page) pairs ensured each page counted only once per user. This is the deduplication-before-aggregation pattern.

IQ-22 What is the difference between sortBy() and sortByKey() with a real example?

sortByKey() only works on PairRDDs and sorts by the key element: [('C002',450), ('C001',300)].sortByKey() -> [('C001',300),('C002',450)]. sortBy() takes a lambda that extracts the sort key: rdd.sortBy(lambda x: x[1], ascending=False) sorts by the VALUE (amount). sortBy(lambda x: len(x[1])) sorts by the LENGTH of a list value. sortByKey() is a special case of sortBy(lambda x: x[0]).

IQ-23 Explain aggregateByKey with a real-world example different from average.

Finding min and max temperature per city simultaneously: zeroValue=(float('inf'),float('-inf')), seqOp=lambda acc,v:(min(acc[0],v), max(acc[1],v)), combOp=lambda a1,a2: (min(a1[0],a2[0]), max(a1[1],a2[1])). This is impossible with reduceByKey (which can only track one value per key) but natural with aggregateByKey (which tracks a 2-tuple accumulator).

IQ-24 What is the difference between filter() on an RDD and a PairRDD?

Functionally identical — filter() returns elements where the lambda returns True. On a plain RDD of integers, lambda x: x>5 tests the value. On a PairRDD of (key,value) tuples, the lambda receives the TUPLE: lambda kv: kv[1]>5 tests the value, lambda kv:

kv[0]=='Delhi' tests the key. filter() is always a narrow transformation regardless of RDD type.

IQ-25 How is mapValues() different from map() on a PairRDD?

On a PairRDD, map(lambda x: ...) receives the entire (key,value) tuple and you must handle both key and value in your lambda. mapValues(lambda v: ...) only receives the VALUE — the key is preserved automatically. mapValues() is cleaner and slightly more efficient because Spark knows the keys don't change (no need to re-partition). Use mapValues() whenever you want to transform values without changing keys.

IQ-26 What happens if you call collect() on a 10 TB RDD?

The Driver node tries to pull ALL 10 TB into its JVM heap memory. This immediately causes an OutOfMemoryError (OOM) and crashes the Spark application. The Driver is typically configured with 4-16 GB of RAM — nowhere near 10 TB. NEVER use collect() on large RDDs. Use take(N) for top-N results, saveAsTextFile() or write.parquet() to persist to storage, or further reduce the data with filter/reduceByKey before collect().

Sateesh Says 💡

You have now covered every operation from Sateesh's PySpark RDD-02 class session — in expert depth.

Step 1: Re-create all three use cases from scratch without looking at the solution. Time yourself.

Step 2: Modify Use Case 2 to also calculate the AVERAGE reading value for abnormal machines — use aggregateByKey!

Step 3: Extend the word count assignment to output only words that appear more than once.

Step 4: Profile the performance — run groupByKey and reduceByKey on the same dataset and compare execution time in Spark UI.

All the best! — Sateesh Pabbathi, MCT | www.levelupedu.net